Laporan TBFO
IF2224 - Teori Bahasa Formal dan Otomata

Disusun Oleh:

Geraldo Artemius - 13524005
Mikhael Andrian Yonatan - 13524051
Junior Natra Situmorang - 13524055
Reynard Nathanael - 13524103
Nicholas Luis Chandra - 13524105

DAFTAR ISI

DAFTAR ISI **2**
BAB I **3**
TEORI SINGKAT **3**
1.1. Lexical Analysis 3
1.2. Deterministic Finite Automata (DFA) 4
BAB II **5**
PERANCANGAN DAN IMPLEMENTASI **5**
2.1. Penerapan DFA 5
2.2. Struktur Direktori 6
2.3. Implementasi Program 7
BAB III **17**
PENGUJIAN **17**
BAB IV **20**
KESIMPULAN DAN SARAN **20**
BAB V **21**
LAMPIRAN **21**
BAB VI **22**
REFERENSI **22**

BAB I
TEORI SINGKAT

1.1. Lexical Analysis

Analisis leksikal (lexical analysis), yang juga dikenal sebagai scanning, merupakan tahap pertama dalam proses kompilasi atau interpretasi bahasa pemrograman. Pada tahap ini, compiler atau interpreter membaca kode sumber yang pada dasarnya merupakan rangkaian karakter, kemudian mengelompokkannya menjadi unit-unit bermakna yang disebut token. **Token merupakan unit terkecil yang memiliki arti dalam sebuah program**, seperti kata kunci (keyword), identifier, operator, konstanta, maupun tanda baca.

Tujuan utama dari analisis leksikal adalah untuk mempermudah tahap berikutnya, yaitu analisis sintaksis (*parsing*). Dengan mengubah karakter mentah menjadi token yang terstruktur, proses parsing dapat dilakukan dengan lebih efisien dalam memahami susunan logis suatu program. Selain itu, analisis leksikal juga berperan dalam menghilangkan elemen yang tidak diperlukan seperti spasi, tab, dan komentar, serta mendeteksi kesalahan awal, misalnya adanya simbol yang tidak dikenali.

Dalam konteks ini, token dapat dipahami sebagai representasi dari pola karakter tertentu yang memiliki makna. Setiap token umumnya terdiri dari dua komponen utama, yaitu jenis token (*token type*) dan nilai token (*token value*). Jenis token menunjukkan kategori seperti identifier, keyword, operator, atau konstanta, sedangkan nilai token menyimpan informasi spesifik dari token tersebut. Selain itu, dikenal juga istilah *lexeme*, yaitu bentuk konkret dari token dalam kode sumber.

Proses analisis leksikal dilakukan oleh komponen yang disebut *lexer* atau *lexical analyzer*. Lexer bertugas membaca kode sumber dari kiri ke kanan, kemudian menggunakan pola tertentu untuk mengenali berbagai jenis token. Ketika suatu rangkaian karakter sesuai dengan pola yang telah ditentukan, lexer akan menghasilkan token yang terdiri dari jenis dan nilai token tersebut.

Sebagai contoh, hasil proses tokenisasi dapat berupa:

- IDENT (a)
- BECOMES
- INTCON (5)

Agar proses pengenalan token dapat dilakukan secara sistematis dan otomatis oleh komputer, mekanisme lexer umumnya diimplementasikan menggunakan konsep *Finite Automata*, khususnya *Deterministic Finite Automata* (DFA).

1.2. Deterministic Finite Automata (DFA)

Deterministic Finite Automata (DFA) merupakan salah satu jenis finite automata yang digunakan untuk mengenali pola dalam suatu rangkaian input. DFA bekerja dengan membaca simbol input satu per satu dan berpindah dari satu state ke state lainnya berdasarkan fungsi transisi yang telah ditentukan. Jika setelah seluruh input diproses mesin berada pada state akhir (accepting state), maka input tersebut diterima; jika tidak, maka ditolak.

Secara formal, DFA didefinisikan sebagai sebuah 5-tuple:

$$(Q, \Sigma, q0, F, \delta)$$

dengan:

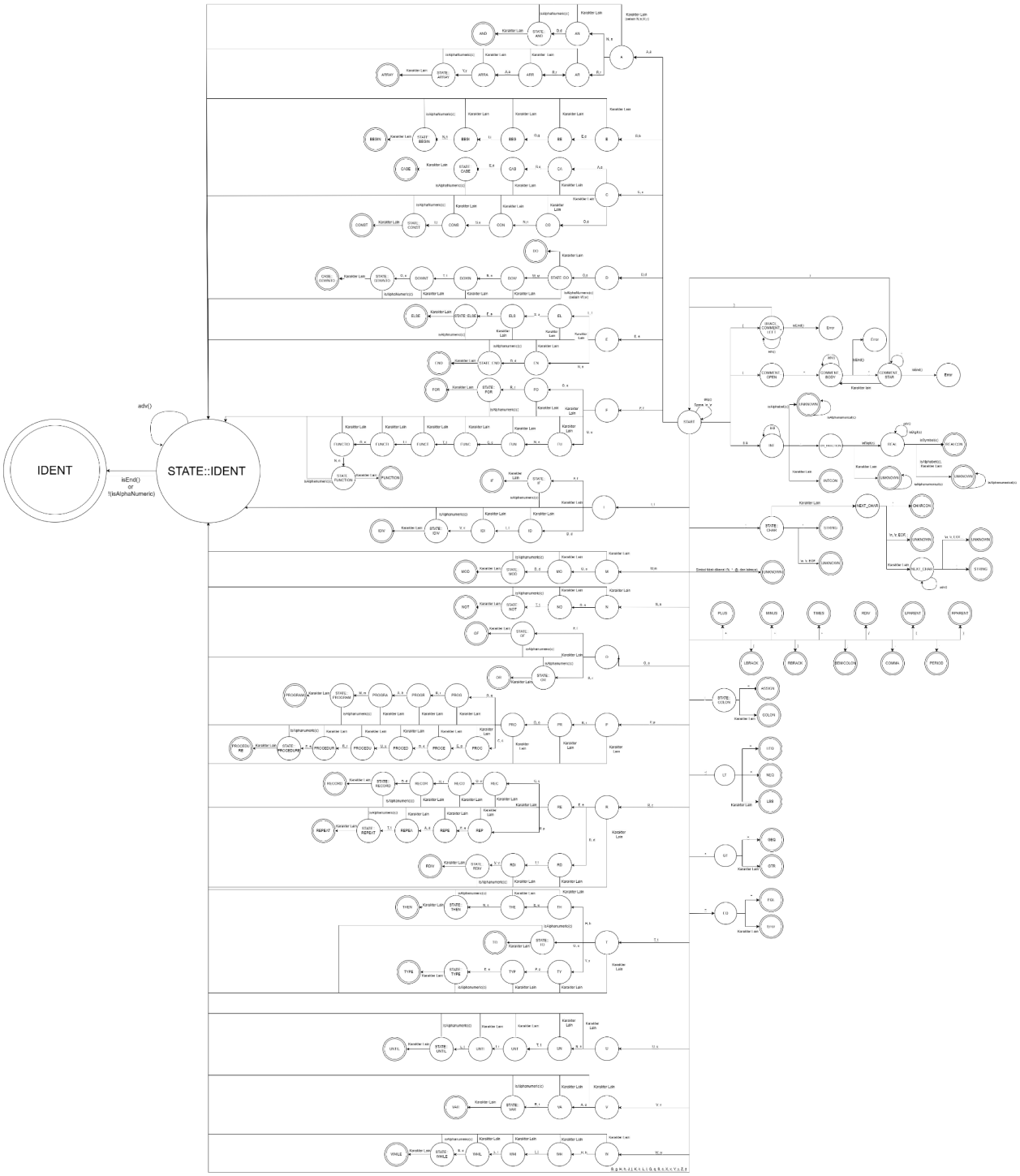
- Q adalah himpunan hingga dari state
- Σ adalah himpunan simbol input (alfabet)
- q0 adalah state awal
- F adalah himpunan state akhir (accepting states)
- δ adalah fungsi transisi, yaitu $\delta : Q \times \Sigma \rightarrow Q$

Ciri utama dari DFA adalah sifat deterministiknya, yaitu untuk setiap pasangan state dan simbol input, hanya terdapat satu kemungkinan state tujuan. Dengan kata lain, tidak ada ambiguitas dalam perpindahan state. Selain itu, DFA tidak memperbolehkan adanya transisi tanpa input (ε-transition), sehingga setiap simbol input harus memiliki definisi transisi yang jelas pada setiap state.

DFA banyak digunakan dalam bidang ilmu komputer, khususnya dalam *compiler design* dan *text processing*, termasuk pada tahap analisis leksikal (lexical analysis). Dalam konteks ini, DFA digunakan untuk mengenali pola token berdasarkan aturan tertentu secara efisien dan sistematis.

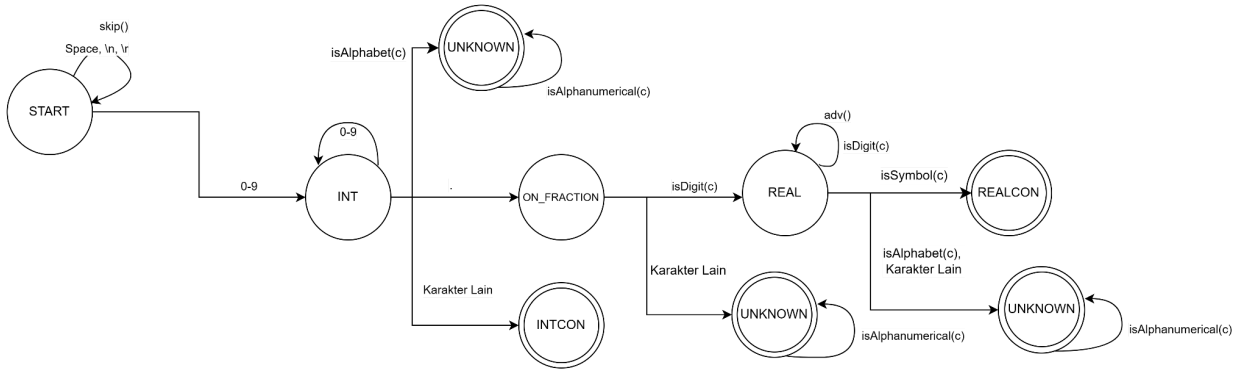
BAB II
PERANCANGAN DAN IMPLEMENTASI

2.1. Diagram DFA



Link : https://drive.google.com/file/d/1cSlXNWUtoq3lUVNd6PH8arFmcnGeKOcB/view?usp=sharing

2.2. Penerapan DFA



Pada program ini, komponen Lexical Analyzer (Lexer) diimplementasikan menggunakan model mesin abstrak Deterministic Finite Automata (DFA). Lexer membaca *string* secara sekuensial, yaitu karakter demi karakter, dan menentukan transisi *state* secara langsung berdasarkan karakter yang sedang dibaca pada saat itu juga tanpa melihat karakter selanjutnya ataupun sebelumnya. Sesuai dengan prinsip dasar Finite Automata yang tidak memiliki memori, program tidak mengumpulkan karakter ke dalam penyimpanan sementara (*buffer*) untuk kemudian dibandingkan nilainya di akhir. Sebagai gantinya, rute transisi *state* digunakan sebagai representasi logis dari pola karakter yang dibaca. Ketika mesin berhasil mencapai suatu *final state*, program dapat secara langsung menyimpulkan token apa yang telah terbentuk murni berdasarkan jalur transisi yang telah dilalui tersebut.

Sifat deterministik pada DFA diterapkan dengan memastikan bahwa setiap *state* memiliki rute transisi yang pasti dan terdefinisi untuk seluruh kemungkinan *input* karakter yang dapat diberikan. Hal ini menjamin bahwa dari *state* manapun mesin berada, pembacaan karakter selanjutnya—baik itu huruf, angka, maupun simbol spesifik—akan selalu membawa mesin ke *state* yang sesuai dengan aturan yang telah didefinisikan.

Sebagai contoh implementasi, pembacaan konstanta angka melibatkan beberapa transisi *state* perantara sebelum mencapai *final state*. Berawal dari *state* START, pembacaan digit angka (0-9) akan memicu transisi ke *state* perantara INT. Dari *state* ini, terdapat beberapa kemungkinan transisi: pembacaan digit angka (0-9) lagi akan membuat mesin *looping* di *state* INT, pembacaan karakter alfabet akan membawa mesin ke *final state* UNKNOWN, dan pembacaan karakter simbol (selain tanda titik) akan menghentikan pembacaan dan membawa mesin ke *final state* INTCON. Apabila mesin membaca tanda titik (.), transisi akan dilanjutkan ke *state* perantara ON_FRACTION untuk mengevaluasi bilangan pecahan. Di dalam *state* ON_FRACTION, terdapat beberapa kemungkinan: pembacaan karakter lain (selain digit angka) membawa mesin ke *final state* UNKNOWN dan pembacaan angka digit (0-9) membawa mesin ke *state* perantara REAL. Di dalam *state* REAL, terdapat beberapa kemungkinan lagi: pembacaan digit angka (0-9) akan membuat mesin *looping* di *state* REAL, pembacaan karakter alfabet membawa mesin ke *final state* UNKNOWN, dan pembacaan karakter simbol akan membawa mesin ke *final state* REALCON. Contoh kasusnya, 123 akan berakhir di *final state* INTCON, 123a akan berakhir di *final state* UNKNOWN, 45.. akan berakhir di *final state* UNKNOWN, 45.b akan berakhir di

final state UNKNOWN, 67.8c akan berakhir di *final state* UNKNOWN, dan 67.8. akan berakhir di *final state* REALCON.

2.3. Struktur Direktori

```
Tubes-IF2224-2026/
|
|__ doc/
|
|__ src/
|  |__ Lexer.cpp
|  |__ Lexer.hpp
|  |__ main.cpp
|  |__ Reader.cpp
|  |__ Reader.hpp
|  |__ Token.cpp
|  |__ Token.hpp
|  |__ Writer.cpp
|  |__ Writer.hpp
|
|__ test/
|
|__ Makefile
|
|__ README.md
```

Secara garis besar,

- Folder `doc` berisi dokumen laporan Tugas Besar dalam bentuk dokumen PDF.
- Folder `src` berisi *source code* program dalam bahasa C++ yang mencakup implementasi mesin DFA (Lexer.cpp, Lexer.hpp), definisi token (Token.cpp, Token.hpp), pemrosesan file (Reader.cpp, Reader.hpp, Writer.cpp, Writer.hpp) dan integrasi seluruh modul (main.cpp).
- Folder `test` berisi file pengujian, baik berupa data masukan (source code bahasa Arion) maupun data keluaran hasil pemrosesan token.
- File `Makefile` berisi kumpulan instruksi kompilasi untuk mempermudah proses build source code C++ menjadi program executable secara otomatis.

2.4. Implementasi Program

1. Token.hpp & Token.cpp

Token.hpp

```
enum TokenType{
    INTCON, // int (1, 3, 48)
    REALCON, // riil (3.14, 26.7)
    CHARCON, // char harus '..' dengan satu karakter ('a', 'b', '1')
    STRING, // sekuens karakter '...' dengan lebih dari satu karakter
('IRK', 'TBFO')
    IDENTIFIER,

    // OPERATOR ARITMATIKA
    PLUS, // +
    MINUS, // -
    TIMES, // *
    IDIV, // div
    RDIV, // pembagian menggunakan '/'
    MOD, // MOD

    // OPERATOR LOGIKA
    AND, // AND
    OR, // OR
    NOT, // NOT

    // COMPARE
    EQL, // = (equal)
    NEQ, // <> (not equal)
    GTR, // > (greater than)
    GEQ, // ≥ (greater than or equal)
    LSS, // < (less than)
    LEQ, // ≤ (less than or equal)

    // ASSIGNMENT SAMA SYMBOL
    ASSIGN, // := , tapi nama tokennya becomes
    LPARENT, // (
    RPARENT, // )
    LBRACK, // [
    RBRACK, // ]
    COMMA, // ,
    SEMICOLON, // ;
    PERIOD, // .
    COLON, // :

    // DEKLARASI
    CONST, // const
    TYPE, // type
    VAR, // var
    FUNCTION, // function
    PROCEDURE, // procedure
```

```
    ARRAY, // array
    RECORD, // record
    PROGRAM, // program

    // DAN LAIN LAIN
    BEGIN, // begin
    END, // end
    IF, // if
    THEN, // then
    ELSE, // else
    CASE, // case
    OF, // of
    REPEAT, // repeat
    UNTIL, // until
    WHILE, // while
    DO, // do
    FOR, // for
    TO, // tp
    DOWNTO, // downto
    COMMENT, // { } atau (*  *)
    UNKNOWN
};

struct Token {
    TokenType type;
    string value;
    int start;
    int end;
};

string typeToString(TokenType type);
```

Pada Token.hpp, implementasi berfungsi sebagai interface utama sistem token. File tersebut mendefinisikan enum `TokenType` yang memuat seluruh kategori token yang dikenali lexer, mulai dari konstanta (integer, real, char, string), identifier, operator aritmatika dan logika, operator perbandingan, kata kunci bahasa seperti const, var, function, begin, if, dan lainnya. Selain itu, didefinisikan juga struktur Token yang menyimpan dua informasi penting, yaitu type untuk jenis token dan value untuk leksem (kata dasar) dari sumber program.

Token.cpp

```
string typeToString(TokenType type) {
    INPUT : Nilai enum TokenType (misalnya INTCON, PLUS, BEGIN, dan
    lain-lain)
    OUTPUT: Nama token dalam bentuk string sesuai format output lexer
```

```
    CARA KERJA:
    1. Periksa nilai type menggunakan percabangan switch-case
    2. Untuk setiap TokenType, kembalikan string representasi yang
    sesuai
    3. Jika tidak ada case yang cocok, kembalikan "unknown"
}
```

Token.cpp merupakan implementasi yang difokuskan pada fungsi `typeToString` yang mengkonversi nilai `TokenType` menjadi label string sesuai format keluaran analisis leksikal. Konversi dilakukan menggunakan switch-case agar setiap jenis token memiliki nama output yang konsisten, misalnya `IDENTIFIER` menjadi ident, `ASSIGN` menjadi `becomes`, dan kata kunci seperti `BEGIN` menjadi `beginsy`. Di bagian akhir, nilai default `unknown` disediakan sebagai fallback jika ada tipe token yang belum dipetakan. Secara keseluruhan, Token.hpp menangani definisi data token, sedangkan Token.cpp menangani representasi teksnya, sehingga alur dari hasil lexer ke output akhir menjadi lebih terstruktur dan konsisten.

2. Lexer.cpp & Lexer.hpp

Lexer.hpp

```
class Lexer {
private :
    enum class State
    {
        START,
        IDENT,
        INT,
        REAL,
        STRING,
        QUOTE_END,
        MINUS,
        UNKNOWN,

        COLON,
        LT, // smaller
        GT, // greater
        EQ, // equal
        CHAR,
        NEXT_CHAR,

        A,
        AN,
        AND,
        AR,
```

```
ARR,
ARRA,
ARRAY,

B,
BE,
BEG,
BEGI,
BEGIN,

C,
CA,
CAS,
CASE,
CO,
CON,
CONS,
CONST,

D,
DI,
DIV,
DO,
DOW,
DOWN,
DOWNT,
DOWNTO,

E,
EL,
ELS,
ELSE,
EN,
END,

F,
FO,
FOR,
FUN,
FUNC,
FUNCT,
FUNCTI,
FUNCTIO,
FUNCTION,

I,
IF,
```

```
M,
MO,
MOD,

N,
NO,
NOT,

O,
OF,
OR,

P,
PR,
PRO,
PROG,
PROGR,
PROGRA,
PROGRAM,
PROC,
PROCE,
PROCED,
PROCEDU,
PROCEDUR,
PROCEDURE,

R,
RE,
REC,
RECO,
RECOR,
RECORD,
REP,
REPE,
REPEA,
REPEAT,

T,
TH,
THE,
THEN,
TO,
TY,
TYP,
TYPE,

U,
UN,
```

```
        UNT,
        UNTI,
        UNTIL,

        V,
        VA,
        VAR,

        W,
        WH,
        WHI,
        WHIL,
        WHILE,


    };

    enum class Skip{
        IGNORE,
        BRACE_COMMENT_LEFT,
        COMMENT_OPEN,
        COMMENT_BODY,
        COMMENT_STAR
    };


    string line;

    int pos;

    char current();
    char adv ();
    bool isEnd();
    void skip();
    bool isAlphabet(char c);
    bool isDigit(char c);
    bool isAlphanumeric(char c);
    bool isSpace(char c);
    bool isSymbol(char c);
    char toLowerChar(char c);
    Token makeToken(TokenType type, int start, int end);
    Token nextToken();

public:
    Lexer(const string& input);
    vector<Token> tokenize();
};
```

Lexer.cpp

```
Lexer::Lexer(const string& input) {
    INPUT : Source code dalam bentuk string
    OUTPUT: Objek lexer dengan posisi awal di indeks 0
    CARA KERJA:
    1. Simpan source code ke variabel internal
    2. Inisialisasi posisi pembacaan ke awal
}

char Lexer::current() {
    INPUT : Posisi lexer saat ini
    OUTPUT: Karakter pada posisi sekarang, atau '\0' jika akhir input
    CARA KERJA:
    1. Cek apakah sudah akhir input
    2. Jika belum, ambil karakter pada posisi sekarang
}

char Lexer::adv() {
    INPUT : Posisi lexer saat ini
    OUTPUT: Karakter saat ini lalu posisi maju 1 langkah
    CARA KERJA:
    1. Cek apakah sudah akhir input
    2. Jika belum, ambil karakter lalu increment posisi
}

bool Lexer::isEnd() {
    INPUT : Posisi lexer dan panjang input
    OUTPUT: true jika posisi sudah melewati/equal panjang input, false
    jika belum
    CARA KERJA:
    1. Bandingkan posisi saat ini dengan ukuran string input
}

bool Lexer::isSymbol(char c) {
    INPUT : Satu karakter c
    OUTPUT: true jika c termasuk simbol bahasa, false jika bukan
    CARA KERJA:
    1. Cocokkan karakter dengan daftar simbol yang didukung lexer
}

bool Lexer::isAlphabet(char c) {
    INPUT : Satu karakter c
    OUTPUT: true jika huruf (a-z/A-Z), false jika bukan
    CARA KERJA:
    1. Cek rentang ASCII huruf kecil dan huruf besar
}
```

```
bool Lexer::isDigit(char c) {
    INPUT : Satu karakter c
    OUTPUT: true jika digit (0-9), false jika bukan
    CARA KERJA:
    1. Cek apakah karakter berada pada rentang '0' sampai '9'
}

bool Lexer::isAlphanumeric(char c) {
    INPUT : Satu karakter c
    OUTPUT: true jika huruf atau digit, false jika bukan
    CARA KERJA:
    1. Gunakan isAlphabet(c) atau isDigit(c)
}

bool Lexer::isSpace(char c) {
    INPUT : Satu karakter c
    OUTPUT: true jika whitespace (spasi/tab/newline/carriage return),
    false jika bukan
    CARA KERJA:
    1. Cocokkan karakter dengan jenis whitespace yang didukung
}

char Lexer::toLowerChar(char c) {
    INPUT : Satu karakter c
    OUTPUT: Versi lowercase dari c (jika uppercase), atau c asli
    CARA KERJA:
    1. Jika huruf besar, konversi ke huruf kecil
    2. Jika bukan huruf besar, kembalikan apa adanya
}

Token Lexer::makeToken(TokenType type, int start, int end) {
    INPUT : Tipe token, indeks awal, indeks akhir
    OUTPUT: Objek Token berisi type, lexeme, start, end
    CARA KERJA:
    1. Ambil substring input dari start sampai end
    2. Bentuk struct Token dari data tersebut
}

void Lexer::skip() {
    INPUT : Posisi lexer saat ini
    OUTPUT: Posisi berpindah melewati whitespace dan komentar
    CARA KERJA:
    1. Lewati seluruh karakter spasi
    2. Jika menemukan komentar { ... }, lewati sampai penutup
    3. Jika menemukan komentar (* ... *), lewati sampai penutup
    4. Jika komentar tidak tertutup, lempar error
}
```

```
vector<Token> Lexer::tokenize() {
    INPUT : Seluruh source code pada objek lexer
    OUTPUT: Vector berisi seluruh token hasil scanning
    CARA KERJA:
    1. Ulangi proses selama belum EOF
    2. Panggil skip() untuk melewati non-token
    3. Panggil nextToken() untuk mengambil token berikutnya
    4. Simpan token ke vector hasil
}

Token Lexer::nextToken() {
    INPUT : Posisi lexer saat ini
    OUTPUT: Satu token berikutnya berdasarkan aturan lexical
    CARA KERJA:
    1. Tentukan state awal
    2. Baca karakter per karakter sesuai transisi state
    3. Kenali identifier/keyword, angka, literal, operator, dan simbol
    4. Kembalikan token yang sesuai atau UNKNOWN jika tidak valid
}
```

3. Reader.cpp & Reader.hpp

Reader.hpp

```
string readFile(const string& filepath);
```

Reader.cpp

```
string readFile(const string& filepath) {
    INPUT : Path file input
    OUTPUT: Seluruh isi file dalam bentuk string tunggal
    CARA KERJA:
    1. Buka file input
    2. Baca seluruh isi file sekaligus ke dalam stringstream
}
```

Reader.cpp memiliki fungsi untuk pembacaan file input secara efisien. Modul ini menggunakan stringstream untuk membaca seluruh isi file .txt sekaligus menjadi satu kesatuan string mentah. Pendekatan ini mempertahankan seluruh karakter asli, termasuk karakter *newline* (\n), sehingga struktur baris kode tetap utuh saat diteruskan ke mesin Lexer.

4. Writer.cpp & Writer.hpp

Writer.hpp

```
void writeTokens(const string& filepath, const vector<Token>& tokens,
const string& sourceCode);

#endif
```

Writer.cpp

```
void writeTokens(const string& filepath, const vector<Token>& tokens,
const string& sourceCode) {
    INPUT : Path file target, daftar token, dan source code asli
    OUTPUT: Tidak ada nilai kembalian (void), menghasilkan file .txt
    berisi format hasil token
    CARA KERJA:
    1. Baca daftar token
    2. Tulis ke file output .txt
}
```

Writer.cpp memiliki fungsi untuk menulis hasil analisis leksikal ke dalam file output .txt. Modul ini mentraversal daftar token untuk mencetak label tipe dan nilai (*lexeme*) sesuai aturan yang telah didefinisikan.

BAB III
PENGUJIAN

1. testCase1.txt

   Input :

```
program Test1;

var
   a, b: string;
   c: real;
   d: char;
   e, f, i: integer;

const
   con: integer;

begin
   con := 10;
   e := 100;
   f := e - 70;
   for(i := 0; i < con; i := i + 1):
      e := e div f;
      e := e + 50;
      e := e mod 2;
      if (e == f):
         e = e / 2;

   while(e <> 0):
      e--;
end.
```

   Output :

```
programsy
ident (Test1)
semicolon

varsy
ident (a)
comma
ident (b)
colon
ident (string)
semicolon
ident (c)
colon
ident (real)
semicolon
ident (d)
colon
ident (char)
semicolon
ident (e)
comma
ident (f)
comma
ident (i)
colon
ident (integer)
semicolon

constsy
ident (con)
colon
ident (integer)
semicolon

beginsy
ident (con)
becomes
intcon (10)
semicolon
ident (e)
becomes
intcon (100)
semicolon
ident (f)
becomes
ident (e)
minus
intcon (70)
semicolon
forsy
lparent
ident (i)
becomes
intcon (0)
semicolon
```

```
intcon (0)
semicolon
ident (i)
lss
ident (con)
semicolon
ident (i)
becomes
ident (i)
plus
intcon (1)
rparent
colon
ident (e)
becomes
ident (e)
idiv
ident (f)
semicolon
ident (e)
becomes
ident (e)
plus
intcon (50)
semicolon
ident (e)
becomes
ident (e)
imod
intcon (2)
semicolon
ifsy
lparent
ident (e)
unknown (==)
ident (f)
rparent
colon
ident (e)
unknown (=)
ident (e)
rdiv
intcon (2)
semicolon

whilesy
lparent
ident (e)
neq
intcon (0)
rparent
colon
ident (e)
minus
minus
semicolon
endsy
```

2. testCase2.txt

Input :

```
program Test2;

var
   a, b, c: string;
   c: real;
   d: char;
   e, f, i, j: integer;
   arr: array of integer;

const
   con: real;

{ Fungsi untuk sorting array }
function sort(array a, int n):
   for(i := 0; i < n; i := i + 1):
   idx := -1;
   max := arr[i];

      for(j := ; j < n; j := j + 1):
         if (arr[j] > max) then
            max := arr[j];
            idx := j;

   if(idx <> -1) then
      arr[j] := arr[idx];
      arr[i] := max;

begin
   con := 10.02;
   a := 'Geraldo';
   b := 'Lapar';
   d := 'L';
   arr[0] := 1.0;
   arr[1] := 2.0;
   arr[3] := 1.2;

   sort(arr, 3);

   c := a + ' ' + b + c + 'Banget';
   if (not(con) AND arr[0] + 1.0 == 2.0) then:
   c := 'terhebat';

   else if (a == 'Geraldo' OR arr[1] > con) then:
      arr[1] := arr[1] * 5.0;
end.
```

Output :

```
programsy
ident (Test2)
semicolon

varsy
ident (a)
comma
ident (b)
comma
ident (c)
colon
ident (string)
semicolon
ident (c)
colon
ident (real)
semicolon
ident (d)
colon
ident (char)
semicolon
ident (e)
comma
ident (f)
comma
ident (i)
comma
ident (j)
colon
ident (integer)
semicolon
ident (arr)
colon
arraysy
ofsy
ident (integer)
semicolon

constsy
ident (con)
colon
ident (real)
semicolon
functionsy
ident (sort)
lparent
arraysy
ident (a)
comma
ident (int)
ident (n)
rparent
colon
forsy
lparent
ident (i)
becomes
intcon (0)
semicolon
ident (i)
lss
ident (n)
semicolon
ident (i)
becomes
ident (i)
plus
lss
ident (n)
semicolon
ident (i)
becomes
ident (i)
plus
intcon (1)
rparent
colon
ident (idx)
becomes
intcon (-1)
semicolon
ident (max)
becomes
ident (arr)
lbrack
ident (i)
rbrack
semicolon

forsy
lparent
ident (j)
becomes
semicolon
ident (j)
lss
ident (n)
semicolon
ident (j)
becomes
ident (j)
plus
intcon (1)
rparent
colon
ifsy
lparent
ident (arr)
lbrack
ident (j)
rbrack
gtr
ident (max)
rparent
thensy
ident (max)
becomes
ident (arr)
lbrack
ident (j)
rbrack
semicolon
ident (idx)
becomes
ident (j)
semicolon

ifsy
lparent
ident (idx)
neq
intcon (-1)
```


```
ifsy
lparent
ident (idx)
neq
intcon (-1)
rparent
thensy
ident (arr)
lbrack
ident (j)
rbrack
becomes
ident (arr)
lbrack
ident (idx)
rbrack
semicolon
ident (arr)
lbrack
ident (i)
rbrack
becomes
ident (max)
semicolon
```


```
beginsy
ident (con)
becomes
realcon (10.02)
semicolon
ident (a)
becomes
string ('Geraldo')
semicolon
ident (b)
becomes
string ('Lapar')
semicolon
ident (d)
becomes
charcon ('L')
semicolon
ident (arr)
lbrack
intcon (0)
rbrack
becomes
realcon (1.0)
semicolon
ident (arr)
lbrack
intcon (1)
rbrack
becomes
realcon (2.0)
semicolon
ident (arr)
lbrack
intcon (3)
rbrack
becomes
realcon (1.2)
semicolon

ident (sort)
lparent
```


```
ident (sort)
lparent
ident (arr)
comma
intcon (3)
rparent
semicolon
```


```
ident (c)
becomes
ident (a)
plus
charcon (' ')
plus
ident (b)
plus
ident (c)
plus
string ('Banget')
semicolon
ifsy
lparent
notsy
lparent
ident (con)
rparent
andsy
ident (arr)
lbrack
intcon (0)
rbrack
plus
realcon (1.0)
unknown (==)
realcon (2.0)
rparent
thensy
colon
ident (c)
becomes
string ('terhebat')
semicolon

elsesy
ifsy
lparent
ident (a)
unknown (==)
string ('Geraldo')
orsy
ident (arr)
lbrack
intcon (1)
rbrack
gtr
ident (con)
rparent
thensy
colon
ident (arr)
lbrack
intcon (1)
rbrack
becomes
ident (arr)
```

```
lbrack
intcon (1)
rbrack
times
realcon (5.0)
semicolon
endsy
period
```

3. testCase3.txt

   Input :

```
program Test3;

var:
    4cho, cho, ending: string
    abcdaco, i: integer
    repeatCoy: string;

procedure printString(string 4cho):
    print(4cho);

begin
    abcdaco := 10;
    i := 0;
    cho := 'Pria_Sawit@PriaSolo.co.id';;
    while (i <= abcdaco) do
        printString(cho);
        i := i + 1;

    repeatCoy := '@';
    repeatCoy := '%';
    repeatCoy := 'SAYA MEMBACA ''KBBI''';
    repeatCoy := '~~~';
    repeatCoy := '''';
    repeatCoy := ''a'';

    repeat (i >= abcdaco) until
        print(repeatCoy);
        i := i / 2;

    ending := '11'
end.
```

   Output :

```
programsy
ident (Test3)
semicolon

varsy
colon
unknown (4cho)
comma
ident (cho)
comma
ident (ending)
colon
ident (string)
ident (abcdaco)
comma
ident (i)
colon
ident (integer)
ident (repeatCoy)
colon
ident (string)
semicolon

proceduresy
ident (printString)
lparent
ident (string)
unknown (4cho)
rparent
colon
ident (print)
lparent
unknown (4cho)
rparent
semicolon

beginsy
ident (abcdaco)
becomes
intcon (10)
semicolon
ident (i)
becomes
intcon (0)
semicolon
ident (cho)
becomes
string ('Pria_Sawit@PriaSolo.co.id')
semicolon
semicolon
whilesy
lparent
ident (i)
leq
ident (abcdaco)
rparent
dosy
ident (printString)
lparent
ident (cho)
rparent
semicolon
ident (i)
becomes
ident (i)
plus
intcon (1)
semicolon

ident (repeatCoy)
becomes
charcon ('@')
semicolon
ident (repeatCoy)
becomes
charcon ('%')
semicolon
ident (repeatCoy)
becomes
string ('SAYA MEMBACA ')
string ('KBBI')
string ('')
semicolon
ident (repeatCoy)
becomes
string ('~~~')
semicolon
ident (repeatCoy)
becomes
string ('')
unknown (')
semicolon
ident (repeatCoy)
becomes
string ('')
ident (a)
string ('')
semicolon

repeatsy
lparent
ident (i)
geq
ident (abcdaco)
rparent
untilsy
ident (print)
lparent
ident (repeatCoy)
rparent
semicolon
ident (i)
becomes
ident (i)
rdiv
intcon (2)
semicolon

ident (ending)
becomes
string ('11')
endsy
period
```

4. testCase4.txt

   Input :

```
program Test4;

type Point(
    x, y: integer;
)

record(
    employee, (id, name);
)

var:
    now, ends: Point;
    i: integer;

begin
    now := Point(0, 0);
    ends := Point(10, 10);
    for i := 10 downto 0 do
        case i mod 2 == 0:
            now.x := now.x + 1;
            now.y := now.y + 1;
        case i mod 2 <> 1:
            ends.x := end.x - 1;
            ends.y := end.y - 1;

    for i := 0 to 10 do
        case i mod 2 <> 0:
            now.x := now.x + 1;
            now.y := now.y + 1;
        case i mod 2 == 1:
            ends.x := end.x - 1;
            ends.y := end.y - 1;
    print('Point = ', x, 'And', y);
end.
```

Output :

5. testCase5.txt

   Input :

```
{ state skip: brace comment }
(*
state skip: paren-star comment
state skip: paren-star comment
state skip: paren-star comment
state skip: paren-star comment
state skip: paren-star comment
state skip: paren-star comment
N1kO TITIT
*)

program p5;
const c1 := 123;
type t1 = record end;
var v1, v2, v3: array[1..3] of integer;

function f1(a1: integer): integer;
begin
    if a1 >= 10 then
        a1 := a1 - 1
    else
        a1 := a1 + 2;
    f1 := a1 div 2 mod 3;
end;

procedure p1;
begin
    repeat
        while v1 < v2 do
            v1 := v1 + 1;
        for v3 := 10 downto 1 do
            v2 := v2 * 2 / 4;
    until v1 = v2;
end;

begin
    v1 := -42;
    v2 := 3.1415;
    v3 := 9abc;

    p1;
    p1();
    p1[1], p1[2];

    and andx
    array arrayx
    begin beginx
    case casex
    const constx
    do dox
    downto downtox
    else elsex
    end endx
    for forx
    function functionx
    if ifx
    mod modx
    not notx
    of ofx
    or orx
    procedure procedurex
    program programx
    record recordx
    repeat repeatx
    then thenx
    to tox
    type typex
```

```
    and andx
    array arrayx
    begin beginx
    case casex
    const constx
    do dox
    downto downtox
    else elsex
    end endx
    for forx
    function functionx
    if ifx
    mod modx
    not notx
    of ofx
    or orx
    procedure procedurex
    program programx
    record recordx
    repeat repeatx
    then thenx
    to tox
    type typex
    until untilx
    var varx
    while whilex

    'a';
    'ab';
    '';
    '''';

    : := < <= <> > >= + - * / ( ) [ ] , ; .
    @ @@
end.
```

Output :

```
programsy
ident (p5)
semicolon
constsy
ident (c1)
becomes
intcon (123)
semicolon
typesy
ident (t1)
unknown (=)
recordsy
endsy
semicolon
varsy
ident (v1)
comma
ident (v2)
comma
ident (v3)
colon
arraysy
lbrack
unknown (1.)
period
intcon (3)
rbrack
ofsy
ident (integer)
semicolon

functionsy
ident (f1)
lparent
ident (a1)
colon
ident (integer)
rparent
colon
ident (integer)
semicolon
beginsy
ifsy
ident (a1)
geq
intcon (10)
thensy
ident (a1)
becomes
ident (a1)
minus
intcon (1)
elsesy
ident (a1)
becomes
ident (a1)
plus
intcon (2)
semicolon
ident (f1)
becomes
ident (a1)
idiv
intcon (2)
imod
intcon (3)
semicolon
```

```

proceduresy
ident (p1)
semicolon
beginsy
repeatsy
whilesy
ident (v1)
lss
ident (v2)
dosy
ident (v1)
becomes
ident (v1)
plus
intcon (1)
semicolon
forsy
ident (v3)
becomes
intcon (10)
downtosy
intcon (1)
dosy
ident (v2)
becomes
ident (v2)
times
intcon (2)
rdiv
intcon (4)
semicolon
untilsy
ident (v1)
unknown (=)
ident (v2)
semicolon
endsy
semicolon

beginsy
ident (v1)
becomes
intcon (-42)
semicolon
ident (v2)
becomes
realcon (3.1415)
semicolon
ident (v3)
becomes
unknown (9abc)
semicolon

ident (p1)
semicolon
ident (p1)
lparent
rparent
semicolon
ident (p1)
lbrack
intcon (1)
rbrack
```

```
ident (p1)
semicolon
ident (p1)
lparent
rparent
semicolon
ident (p1)
lbrack
intcon (1)
rbrack
comma
ident (p1)
lbrack
intcon (2)
rbrack
semicolon

andsy
ident (andx)
arraysy
ident (arrayx)
beginsy
ident (beginx)
casesy
ident (casex)
constsy
ident (constx)
dosy
ident (dox)
downtosy
ident (downtox)
elsesy
ident (elsex)
endsy
ident (endx)
forsy
ident (forx)
functionsy
ident (functionx)
ifsy
ident (ifx)
imod
ident (modx)
notsy
ident (notx)
ofsy
ident (ofx)
orsy
ident (orx)
proceduresy
ident (procedurex)
programsy
ident (programx)
recordsy
ident (recordx)
repeatsy
ident (repeatx)
thensy
ident (thenx)
tosy
ident (tox)
typesy
ident (typex)
untilsy
ident (untilx)
```

```
proceduresy
ident (procedurex)
programsy
ident (programx)
recordsy
ident (recordx)
repeatsy
ident (repeatx)
thensy
ident (thenx)
tosy
ident (tox)
typesy
ident (typex)
untilsy
ident (untilx)
varsy
ident (varx)
whilesy
ident (whilex)

charcon ('a')
semicolon
string ('ab')
semicolon
string ('')
semicolon
string ('')
unknown (')
semicolon

colon
becomes
lss
leq
neq
gtr
geq
plus
minus
times
rdiv
lparent
rparent
lbrack
rbrack
comma
semicolon
period
unknown (@)
unknown (@@
end)
period
```

BAB IV
KESIMPULAN DAN SARAN

4.1. Kesimpulan

Berdasarkan perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa *Lexical Analyzer* (Scanner) untuk subset bahasa Pascal telah berhasil dibangun dan direpresentasikan secara komprehensif melalui *Deterministic Finite Automaton* (DFA). *Lexer* mampu memindai karakter input secara sekuensial dan mengelompokkannya menjadi token-token yang terstruktur. Program telah berhasil menangani berbagai spesifikasi seperti pengenalan *identifier*, pembacaan konstanta numerik ganda (*integer* dan *real*), penanganan *string* dan karakter, pengenalan operator seperti *assignment* (`:=`), deteksi *keyword* secara *case-insensitive,* serta melakukan sebagian *error handling* untuk beberapa state.

Implementasi yang telah berjalan mencakup fungsi-fungsi dasar scanner, yaitu:

1. Pengenalan karakter dasar seperti huruf, digit, simbol, dan whitespace.
2. Pembentukan token dengan informasi tipe token, nilai lexeme, serta posisi awal dan akhir.
3. Pengenalan token operator dan delimiter utama, seperti +, -, *, /, (,), [,], ,, ;, ..
4. Penanganan komentar dan karakter non-signifikan melalui mekanisme skip, termasuk komentar dalam Pascal.
5. Pengenalan literal karakter/string sederhana, bilangan, serta token tak dikenal (unknown) untuk input yang tidak valid.
6. Konversi tipe token ke representasi string untuk kebutuhan output hasil analisis.
7. Arsitektur program telah terpisah dengan baik antara modul pembaca file, lexer, token mapping, dan penulis output, sehingga alur analisis leksikal menjadi lebih modular dan mudah dikembangkan.

4.2. Saran

Untuk pengembangan selanjutnya, disarankan memperkuat *error recovery* pada Lexer dengan mencatat posisi *error*, mengabaikan sekuensial karakter yang bermasalah, dan tetap melanjutkan pemindaian. Selanjutnya, dapat menyempurnakan seluruh transisi state DFA pada proses nextToken agar semua state yang telah dirancang benar-benar terimplementasi konsisten. Perlu dilakukan pengujian lebih lanjut dengan variasi kasus normal, batas, dan salah (invalid input), termasuk kombinasi komentar, keyword, identifier, dan simbol berurutan.

BAB V
LAMPIRAN

Link Repository Github: https://github.com/staplesmaster/SSS-Tubes-IF2224-2026
Link Workspace Diagram Transisi DFA:
https://drive.google.com/file/d/1cSlXNWUtoq3lUVNd6PH8arFmcnGeKOcB/view?usp=sharing

Pembagian Tugas:

Nama	NIM	Bagian	Persentase
Geraldo Artemius	13524005	• Kode Program • Diagram DFA • Laporan	20%
Mikhael Andrian Yonatan	13524051	• Kode Program • Laporan	20%
Junior Natra Situmorang	13524055	• Kode Program • Laporan	20%
Reynard Nathanael	13524103	• Kode Program • Diagram DFA • Laporan	20%
Nicholas Luis Chandra	13524105	• Kode Program • Laporan	20%

BAB VI
REFERENSI

[1] GeeksforGeeks, “Introduction of Lexical Analysis,” *GeeksforGeeks*, Jul. 13, 2015. Available: https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/

[2] GeeksforGeeks, “Introduction of Finite Automata,” *GeeksforGeeks*, Jul. 09, 2015. Available: https://www.geeksforgeeks.org/theory-of-computation/introduction-of-finite-automata/

[3] GeeksforGeeks, “Lexical Analyzer in C++,” GeeksforGeeks, Jul. 03, 2024. Available: https://www.geeksforgeeks.org/cpp/lexical-analyzer-in-cpp/

[4] Santoso, Judhi, “Deterministic /Non Deterministic Automata,” Edunex, Mar. 29, 2026. Available: https://edunexcontentprodhot.blob.core.windows.net/edunex/2025/81979-Formal-Language-Theory-and-Automata/362555-Module-02/file/1757509229813_slide-02?sv=2024-11-04&spr=https&st=2025-09-10T13%3A00%3A29Z&se=2027-09-10T13%3A00%3A29Z&sr=b&sp=r&sig=RD5%2FQ6gfJe6r4PsGZc6RarddExwW1oPA13U7MPW1JHM%3D&rsct=application%2Fpdf

[5] "Google Docs document," Google Docs. https://docs.google.com/document/d/16dKSX_N3_pLFDT-C9qeZBtd-xERblDOmqh3EuuMvhrE/edit (accessed Apr. 2, 2026).